ACADEMIC & TECHNICAL DEFENSE COMPENDIUM

INFORMATION MANAGEMENT: DATABASE ARCHITECTURE & SYSTEM INTEGRATION REVIEWER

System: Community Complaint and Incident Reporting System (Banlic-CCIRS) • **LGU:** Barangay Banlic, Calamba City, Laguna
Course Scope: Relational Modeling, 3NF Normalization, MySQL InnoDB Engine, Laravel Eloquent ORM, and ACID Transaction Flow

MySQL 8 / MariaDB	16 Normalized Tables	Laravel 11 / PHP 8.2+	Asia/Manila (UTC+8)
RDBMS & STORAGE ENGINE (INNODB)	SCHEMA STRUCTURE (3NF / BCNF)	DATA ACCESS TIER (ELOQUENT ORM)	TEMPORAL CONSISTENCY & VALIDATION

1. EXECUTIVE OVERVIEW & COURSE CONTEXT

In **Information Management (IM)**, an enterprise database serves as the authoritative, persistent information backbone that models organizational business logic, maintains accountability, and powers operational reporting. In civic governance, the **Barangay Banlic Community Complaint and Incident Reporting System (Banlic-CCIRS)** automates incident logging, multi-committee assignment, investigation recording, resident ID verification, and formal dispute mediation (Katarungang Pambarangay).

> **Core Course Objective:** To understand how real-world civic entities (residents, grievances, committees, investigators) transform into a mathematical **Relational Schema (3NF)**, how data integrity is enforced via **referential constraints**, and how application software connects to the database via **Object-Relational Mapping (ORM)** and **ACID Transactions**.

1.1 Strategic Database Objectives in Local Governance

- **Eliminating Information Silos:** Replaces disjointed paper logbooks and Excel sheets with a centralized MySQL database, creating a single source of truth across all barangay committees.
- **Auditability & Non-Repudiation:** Chronologically tracks every dispute phase (Pending → Under Review → Assigned → In Progress → Resolved → Closed → Rejected) along with investigator remarks, actor IDs, and timestamps.
- **Data Privacy & Access Control:** Enforces Role-Based Access Control (RBAC) separating administrative staff from residents, and complies with Republic Act 10173 (Philippine Data Privacy Act of 2012) via masked PII and salted bcrypt password hashing.
- **Transactional Durability:** Utilizes the MySQL InnoDB storage engine with Write-Ahead Logging (WAL) and foreign key referential cascading to prevent data corruption and orphaned records.

2. SYSTEM ARCHITECTURE: THE MULTI-TIER INTEGRATION PIPELINE

The Banlic-CCIRS employs a classic **Three-Tier Client-Server Architecture** integrated with the **Model-View-Controller (MVC)** design pattern. This architecture enforces strict separation of concerns, ensuring that database schema changes do not break presentation templates, and user interface modifications never jeopardize data integrity.

1. PRESENTATION TIER (CLIENT BROWSER) — **Blade Views • Tailwind CSS • Alpine.js**
Renders responsive UI: Resident Filing Portal, Admin Dispatch Board, and Real-Time Tracking Timeline. Collects form inputs and dispatches HTTP POST/GET requests secured by CSRF tokens.

↓ HTTP Request / CSRF Token Verification ↓

2. APPLICATION LOGIC TIER (LARAVEL 11 CONTROLLERS) — **Routing • Middleware • Business Validation**
Authorizes user roles, enforces 5-minute clock drift grace tolerance (`before_or_equal:now + 5 mins`), generates unique reference numbers (`BRGY-YYYY-XXXX`), and orchestrates business workflows.

↓ Model Invocation / Transaction Boundary (DB::transaction) ↓

3. DATA ACCESS / ORM TIER (ELOQUENT & PDO) — **Active Record Models • PDO Prepared Statements**
Translates PHP objects to relational SQL queries. Manages model relationships (`belongsTo` , `hasMany` , `hasOne`), attribute casting, and parameter binding preventing SQL injection.

↓ TCP Port 3306 / Unix Socket (pdo_mysql) ↓

4. DATABASE TIER (MYSQL 8 / INNODB ENGINE) — **barangay_complaint_db • utf8mb4_unicode_ci**
Physical storage layer enforcing relational constraints, composite B+Tree indexes, AUTO_INCREMENT keys, row-level locking (MVCC), and Write-Ahead Logging (WAL) ensuring ACID durability.

> **Information Management Principle • Data Independence:** *Physical data independence* is achieved because MySQL storage files (tablespaces, indexes, redo logs) can be migrated or tuned without altering application code. *Logical data independence* is preserved because controllers interact with abstract Eloquent models rather than raw physical table schemas.

3. ENTITY-RELATIONSHIP (ER) CONCEPTUAL & LOGICAL MODELING

The Banlic-CCIRS logical schema comprises **16 normalized relational tables** organized into four distinct operational domains: Reference Lookups, Identity & Access, Dispute Lifecycle, and Security & Audit Trails.

Domain 1 • Reference Lookups (Static Metadata)

roles: (role_id, role_name, description) • 1:N with users

departments: (department_id, department_name, is_active) • Barangay offices/committees

areas: (area_id, area_name, is_active) • Puroks 1 to 5 geographic zones

report_types: (report_type_id, type_name) • Complaint, Concern, Incident

complaint_categories: (category_id, department_id, category_name) • Maps to depts

priority_levels: (priority_id, priority_name, sort_order) • Low to Urgent

complaint_statuses: (status_id, status_name, sort_order, badge_color) • 7 states

Domain 2 • Identity & Resident Verification

users: (user_id, role_id, department_id, area_id, email, password_hash, verification_status, verified_by) • Core actor entity

area_staff: (area_id, user_id, assigned_at) • Associative/junction table for Purok officers (M:N)

password_resets: (reset_id, user_id, token_hash, expires_at) • 1:N with users

password_reset_tokens: (email, token, created_at) • Laravel auth reset

Domain 3 • Core Dispute Operations

complaints: (complaint_id, reference_no, complainant_id, type_id, category_id, priority_id, area_id, status_id, incident_date) • Central hub

complaint_assignments: (assignment_id, complaint_id, assigned_to, assigned_by, is_current) • Officer delegation trail

complaint_status_history: (history_id, complaint_id, old_status_id, new_status_id, changed_by) • Immutable timeline log

investigations: (investigation_id, complaint_id, investigator_id, findings) • Hearing evidence

resolutions: (resolution_id, complaint_id [UNIQUE], resolved_by) • Strict 1:1 settlement

attachments: (attachment_id, complaint_id, investigation_id, file_path) • Photos & docs

Domain 4 • Security & Audit Trail

audit_logs: (log_id, user_id, action, table_name, record_id, ip_address, created_at) • Non-repudiation ledger

notifications: (notification_id, user_id, complaint_id, message, is_read) • User notification bell

3.1 Cardinality & Multiplicity Relationship Matrix

Parent Entity	Child Entity	Cardinality	Foreign Key Column	Referential Action
`users` (Resident)	`complaints`	**1 : N** (One-to-Many)	`complainant_id`	RESTRICT (Cannot delete user with active cases)
`complaints`	`attachments`	**1 : N** (One-to-Many)	`complaint_id`	CASCADE (Deleting complaint deletes files)
`complaints`	`complaint_status_history`	**1 : N** (One-to-Many)	`complaint_id`	CASCADE (History purged on complaint delete)
`complaints`	`complaint_assignments`	**1 : N** (One-to-Many)	`complaint_id`	CASCADE (Assignments purged on delete)
`complaints`	`resolutions`	**1 : 1** (One-to-One)	`complaint_id (UNIQUE)`	CASCADE (Resolution purged on complaint delete)
`areas` & `users`	`area_staff`	**M : N** (Many-to-Many)	`area_id`, `user_id`	CASCADE on both foreign keys (Junction table)
`departments`	`complaint_categories`	**1 : N** (One-to-Many)	`department_id`	SET NULL (Category stays active if dept changes)
`departments`	`users` (Staff)	**1 : N** (One-to-Many)	`department_id`	SET NULL (Staff reassigned if dept dissolves)

4. RELATIONAL DATABASE THEORY & NORMALIZATION ANALYSIS

Database Normalization is the formal mathematical process of decomposing complex table structures to minimize redundancy and eradicate **Insertion, Deletion, and Update Anomalies**. The Banlic-CCIRS schema is certified in **Third Normal Form (3NF)** with Boyce-Codd (BCNF) compliance.

4.1 First Normal Form (1NF) Proof & Verification

- **Atomicity of Attributes:** Every column contains strictly non-divisible, scalar values. Resident names are separated into `first_name`, `middle_name`, and `last_name` rather than a composite string.
- **Elimination of Repeating Groups:** In unnormalized spreadsheets, multiple evidence photos or status remarks are saved as comma-separated lists (e.g., `"img1.jpg, img2.jpg"`). Banlic-CCIRS decouples evidence into the dedicated child table `attachments` and timeline events into `complaint_status_history`.
- **Unique Tuple Identification:** Every table possesses an unambiguous Primary Key (e.g., `complaint_id`, `user_id`).

4.2 Second Normal Form (2NF) Proof & Verification

- **Prerequisite:** The relation is in 1NF.
- **No Partial Functional Dependencies:** Every non-prime attribute must depend on the *entire* primary key, not a subset.
- **Case Study • `area_staff` Table:** The table uses a **composite primary key**: `(area_id, user_id)`. The attribute `assigned_at` depends strictly on the combination of *both* the area and the designated officer, not solely on one column. All single-column primary key tables automatically satisfy 2NF because partial dependency cannot exist on a single key.

4.3 Third Normal Form (3NF) Proof & Verification

- **Prerequisite:** The relation is in 2NF.
- **No Transitive Dependencies:** Non-prime attributes must not depend on other non-prime attributes (X → Y → Z). Attributes must depend *only on the primary key*.
- **Case Study • Decoupling Departments & Categories:** A poorly designed schema might place `department_name` and `department_head` directly inside the `complaints` table. If a department changes its name, updating thousands of complaint rows creates an **Update Anomaly**. Deleting all complaints in an area would wipe out department data (**Deletion Anomaly**). In Banlic-CCIRS: `complaint_id → category_id` and `category_id → department_id`.
  By separating `departments`, `complaint_categories`, `priority_levels`, and `complaint_statuses` into lookup tables, all transitive dependencies are eliminated.

> **Defense High-Yield Question:** *"Why does the complaints table still store status_id when complaint_status_history already logs status changes? Isn't that redundant?"*
> **Model Answer:** It is **controlled operational denormalization**. Keeping the active `status_id` on `complaints` allows O(1) indexed querying on the admin dashboard (`WHERE status_id = 1`) without executing an expensive aggregate subquery (`SELECT MAX(changed_at) FROM complaint_status_history GROUP BY complaint_id`) across millions of historical records. Synchronization is guaranteed via atomic **database transactions**.

5. COMPREHENSIVE DATA DICTIONARY (METADATA CATALOG)

The **Data Dictionary** catalogs physical column specifications, storage domains, nullability, keys, and relational constraints across the system's operational tables.

5.1 Core Entity: `complaints` Table

Column Name	Data Type	Null?	Key / Index	Description & Integrity Constraints
`complaint_id`	INT UNSIGNED	NO	PRIMARY	Surrogate primary key, AUTO_INCREMENT. Internal database identifier.
`reference_no`	VARCHAR(30)	NO	UNIQUE	Format: `BRGY-YYYY-XXXX`. Public identifier used in URLs & resident tracking.
`complainant_id`	INT UNSIGNED	NO	FK users	References `users(user_id)`. Foreign key linking to resident filer.
`report_type_id`	TINYINT UNSIGNED	NO	FK report_types	References `report_types(report_type_id)` (Complaint, Concern, Incident).
`category_id`	INT UNSIGNED	NO	FK categories	References `complaint_categories(category_id)`. Indexed: `idx_comp_category`.
`priority_id`	TINYINT UNSIGNED	NO	FK priorities	References `priority_levels(priority_id)` (Low, Medium, High, Urgent).
`area_id`	INT UNSIGNED	NO	FK areas	References `areas(area_id)` (Purok 1-5). Indexed: `idx_comp_area`.
`status_id`	TINYINT UNSIGNED	NO	FK statuses	References `complaint_statuses(status_id)`. Indexed: `idx_comp_status`.
`title`	VARCHAR(150)	NO	None	Headline or summary statement describing the dispute.
`description`	TEXT	NO	None	Detailed factual narrative submitted by the resident.
`location_details`	VARCHAR(255)	YES	None	Specific street number, house landmark, or zone coordinates within Purok.
`incident_date`	DATETIME	YES	None	Incident timestamp. Validated: ≤ server time + 5-minute clock drift tolerance.
`created_at`	TIMESTAMP	NO	INDEX	Submission timestamp (Default: CURRENT_TIMESTAMP). Indexed: `idx_comp_created`.
`updated_at`	TIMESTAMP	NO	None	Auto-updated timestamp on record modification via ON UPDATE CURRENT_TIMESTAMP.

5.2 Identity & Resident Verification: `users` Table

Column Name	Data Type	Null?	Key / Index	Description & Integrity Constraints
`user_id`	INT UNSIGNED	NO	PRIMARY	Surrogate primary key, AUTO_INCREMENT. System-wide actor ID.
`role_id`	TINYINT UNSIGNED	NO	FK roles	1 = Admin (Barangay Staff/Lupon), 2 = Complainant (Resident).
`department_id`	INT UNSIGNED	YES	FK depts	ON DELETE SET NULL. Assigned barangay committee for administrative users.
`committee_position`	VARCHAR(100)	YES	None	Designated rank (e.g., Committee Chairman, Investigator, Lupon Member).
`area_id`	INT UNSIGNED	YES	FK areas	ON DELETE SET NULL. Resident's registered home Purok.
`first_name`, `last_name`	VARCHAR(60)	NO	None	Legal names. Masked on public interfaces to uphold RA 10173 privacy.
`email`	VARCHAR(120)	NO	UNIQUE	Login credential and notification destination address.
`password_hash`	VARCHAR(255)	NO	None	One-way cryptographic hash generated via bcrypt (Cost Factor: 12).
`verification_status`	VARCHAR(20)	NO	None	Resident ID review state: `pending`, `approved`, `rejected`.
`verification_doc_path`	VARCHAR(255)	YES	None	Filesystem storage path for uploaded government ID or barangay certificate.
`verified_by`	INT UNSIGNED	YES	FK users	ON DELETE SET NULL. Self-referencing FK linking to admin reviewer.

5.3 Supporting Operational Tables

Table Name	Primary Key	Foreign Key Relationships	Functional Role in System
`complaint_assignments`	`assignment_id`	`complaint_id` (CASCADE), `assigned_to` (users), `assigned_by` (users)	Delegates complaints to staff. `is_current=true` flags the active officer.
`complaint_status_history`	`history_id`	`complaint_id` (CASCADE), `old_status_id`, `new_status_id`, `changed_by`	Immutable chronological ledger powering the public progress timeline.
`investigations`	`investigation_id`	`complaint_id` (CASCADE), `investigator_id` (users)	Field investigation findings, witness testimonies, and hearing notes.
`resolutions`	`resolution_id`	`complaint_id` (UNIQUE, CASCADE), `resolved_by` (users)	1:1 settlement agreement, mediation terms, or formal closure minutes.
`attachments`	`attachment_id`	`complaint_id` (CASCADE), `investigation_id` (SET NULL), `uploaded_by`	Metadata catalog for photos, CCTV video, and affidavit documents.
`notifications`	`notification_id`	`user_id` (CASCADE), `complaint_id` (CASCADE)	Real-time alert messages driving the resident's navbar notification bell.
`audit_logs`	`log_id` (BIGINT)	`user_id` (SET NULL)	Security audit ledger logging user IP addresses, actions, and altered tables.

6. HOW THE DATABASE CONNECTS TO THE SYSTEM

A frequent question in defense hearings is: *"How does your Laravel code establish a physical connection to MySQL and execute queries?"* In Banlic-CCIRS, this lifecycle flows through configuration parsing, PDO driver management, schema migrations, and the Eloquent Active Record pattern.

6.1 Connection Pipeline: `.env` → `config/database.php` → PDO

During framework bootstrapping, Laravel reads environment variables from `.env` and instantiates a `PDO` connection pool using the MySQL driver configuration:

```
// .env — Environment Variables
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=barangay_complaint_db
DB_USERNAME=root
DB_PASSWORD=secret
```

```
// config/database.php — Driver Specification
'connections' => [
    'mysql' => [
        'driver'    => 'mysql',
        'host'      => env('DB_HOST', '127.0.0.1'),
        'port'      => env('DB_PORT', '3306'),
        'database'  => env('DB_DATABASE', 'barangay_complaint_db'),
        'username'  => env('DB_USERNAME', 'root'),
        'password'  => env('DB_PASSWORD', ''),
        'charset'   => 'utf8mb4',
        'collation' => 'utf8mb4_unicode_ci',
        'strict'    => true,     // Enforces strict SQL rules (rejects invalid dates and unsafe grouping)
        'engine'    => 'InnoDB',  // Guarantees transactional ACID support
    ],
],
```

6.2 Schema Evolution via Laravel Migrations

Instead of importing unversioned raw SQL dumps manually, table creation and column adjustments are defined in **Laravel Migrations** (e.g., `database/migrations/2026_01_01_000001_create_barangay_tables.php`). This provides repeatable schema deployment and rollback capability:

```
// Defining complaints table schema via Schema Builder
Schema::create('complaints', function (Blueprint $table) {
    $table->increments('complaint_id');                          // INT UNSIGNED AUTO_INCREMENT PK
    $table->string('reference_no', 30)->unique();               // Unique Candidate Key
    $table->unsignedInteger('complainant_id');
    $table->unsignedTinyInteger('status_id')->index();        // Secondary Index on Status
    $table->text('description');
    $table->dateTime('incident_date')->nullable();
    $table->timestamps();

    // Foreign Key Constraints enforcing Referential Integrity
    $table->foreign('complainant_id')->references('user_id')->on('users');
    $table->foreign('status_id')->references('status_id')->on('complaint_statuses');
});
```

6.3 Object-Relational Mapping (Eloquent ORM) Deep Dive

Eloquent maps each relational table to a PHP class implementing the **Active Record Pattern**. Below is the production implementation in `app/Models/Complaint.php`:

```php
// app/Models/Complaint.php — Eloquent Model Mapping
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Complaint extends Model
{
    protected $primaryKey = 'complaint_id';  // Explicitly binds non-default PK column
    protected $guarded = [];                 // Mass assignment configuration

    protected function casts(): array {
        return ['incident_date' => 'datetime']; // Auto-converts SQL DATETIME to Carbon instance
    }

    // Conceals internal auto-increment ID from URLs; binds route parameters to reference_no
    public function getRouteKeyName() { return 'reference_no'; }

    // Declarative Relational Mappings
    public function complainant()      { return $this->belongsTo(User::class, 'complainant_id', 'user_id'); }
    public function status()           { return $this->belongsTo(ComplaintStatus::class, 'status_id', 'status_id'); }
    public function attachments()      { return $this->hasMany(Attachment::class, 'complaint_id', 'complaint_id'); }
    public function history()          { return $this->hasMany(StatusHistory::class, 'complaint_id', 'complaint_id'); }
    public function resolution()       { return $this->hasOne(Resolution::class, 'complaint_id', 'complaint_id'); }
    public function currentAssignment() {
        return $this->hasOne(ComplaintAssignment::class, 'complaint_id', 'complaint_id')
                    ->where('is_current', true)->latestOfMany('assignment_id');
    }
}
```

7. TRANSACTION LIFECYCLE & ACID PROPERTIES

In database management, operations modifying multiple tables must execute atomically. If an error occurs halfway through, partial updates create corrupted, desynchronized records. Banlic-CCIRS enforces integrity through **ACID Transactions**.

ATOMICITY	CONSISTENCY	ISOLATION	DURABILITY
"ALL OR NOTHING" EXECUTION	PRESERVES SCHEMA INVARIANTS	CONCURRENT USER SEPARATION	PERSISTENT REDO LOGS

- **Atomicity (All or Nothing):** Handled via `DB::transaction()`. When a resident files a complaint, multiple database operations execute (insert into `complaints`, store `attachments`, log initial status in `complaint_status_history`, write `audit_logs`). If any step throws an exception, the entire transaction is rolled back (`ROLLBACK`), preventing partial records.
- **Consistency:** Constraints such as foreign key checks, non-null values, and unique constraints (`reference_no`) are strictly enforced by MySQL before changes commit.
- **Isolation:** MySQL InnoDB uses **Multi-Version Concurrency Control (MVCC)** and row-level locks, allowing multiple residents to file complaints simultaneously without race conditions or dirty reads.
- **Durability:** Once committed, updates are written to the InnoDB Write-Ahead Log (WAL / Redo Log) on disk before acknowledging completion, surviving hardware reboots and crashes.

7.1 Production Code Walkthrough: Atomic Filing in `ComplaintController.php`

```php
// app/Http/Controllers/ComplaintController.php — Atomic Transaction Implementation
$complaint = DB::transaction(function () use ($data, $request, $user) {
    // 1. Insert Core Complaint Tuple
    $complaint = Complaint::create([
        'reference_no'   => $this->generateReferenceNumber(), // e.g., BRGY-2026-00014
        'complainant_id' => $user->user_id,
        'report_type_id' => $data['report_type_id'],
        'category_id'    => $data['category_id'],
        'priority_id'    => $data['priority_id'],
        'area_id'        => $data['area_id'],
        'status_id'      => 1, // Default initial status: 'Pending'
        'title'          => $data['title'],
        'description'    => $data['description'],
        'incident_date'  => $data['incident_date'],
    ]);

    // 2. Store Physical Files and Insert Attachment Metadata Tuples
    if ($request->hasFile('attachments')) {
        foreach ($request->file('attachments') as $file) {
            $path = $file->store('complaints', 'public');
            $complaint->attachments()->create([
                'uploaded_by' => $user->user_id,
                'file_name'   => $file->getClientOriginalName(),
                'file_path'   => $path,
                'file_size'   => $file->getSize(),
            ]);
        }
    }

    // 3. Initialize Status History Audit Timeline
    StatusHistory::create([
        'complaint_id'  => $complaint->complaint_id,
        'new_status_id' => 1,
        'changed_by'    => $user->user_id,
        'remarks'       => 'Complaint filed by resident.',
    ]);

    // 4. Write Non-Repudiation Security Audit Log
    AuditLog::create([
        'user_id'     => $user->user_id,
        'action'      => 'CREATE',
        'table_name'  => 'complaints',
        'record_id'   => $complaint->complaint_id,
        'description' => 'Filed complaint ' . $complaint->reference_no,
        'ip_address'  => $request->ip(),
    ]);

    return $complaint; // All operations committed atomically (COMMIT)
});
```

7.2 Status Transition & Case Resolution Transaction (Admin Controller)

When an authorized officer updates the status or records a formal resolution in `Admin\ComplaintController.php`, atomic integrity is similarly enforced:

```php
// app/Http/Controllers/Admin/ComplaintController.php — Status Update Transaction
DB::transaction(function () use ($complaint, $validated, $oldStatusId, $newStatusId, $statusChanged) {
    if ($statusChanged) {
        $complaint->status_id = $newStatusId;
        $complaint->save();

        // Record immutable historical audit entry
        StatusHistory::create([
            'complaint_id'  => $complaint->complaint_id,
            'old_status_id' => $oldStatusId,
            'new_status_id' => $newStatusId,
            'changed_by'    => auth()->id(),
            'remarks'       => $validated['remarks'] ?? null,
        ]);
    }

    // If resolved, enforce 1:1 Resolution record creation
    if ($newStatusId == 5 && !empty($validated['resolution_details'])) {
        Resolution::updateOrCreate(
            ['complaint_id' => $complaint->complaint_id],
            [
                'resolved_by'        => auth()->id(),
                'resolution_details' => $validated['resolution_details'],
                'resolved_at'        => now(),
            ]
        );
    }
});
```

8. DATABASE SECURITY, INTEGRITY & PRIVACY (RA 10173)

- **SQL Injection (SQLi) Prevention:** Handled via **PDO Parameterized Prepared Statements**. Input parameters are sent as separate protocol packets from the SQL execution plan, rendering SQL injection technically impossible.
- **Cryptographic Credential Security:** Passwords are never stored in plaintext. They are hashed using **bcrypt** (Blowfish cipher) with 12 rounds of salting (`BCRYPT_ROUNDS=12`), preventing rainbow table and dictionary attacks.
- **Data Privacy Act Compliance (Resident Masking):** On public administrative displays, sensitive resident identities are masked dynamically via the Eloquent Accessor `getMaskedComplainantAttribute()` : `"Juan Dela Cruz" → "J*** D*** C***"` , protecting victim identities while maintaining case traceability.

9. PHYSICAL PERFORMANCE TUNING & INDEXING STRATEGY

9.1 Strategic B+Tree Indexing

MySQL InnoDB creates a **Clustered Index** on every table's Primary Key. In addition, Banlic-CCIRS provisions targeted **Secondary Indexes** on frequently filtered foreign keys:

- `idx_comp_status (status_id)` : Accelerates status-filtered dashboard queries (e.g., viewing all "Pending" complaints).
- `idx_comp_area (area_id)` : Optimizes geographic complaint filtering by Purok.
- `idx_comp_created (created_at)` : Powers time-series incident trends and monthly barangay analytics reports without table scans.
- `idx_assign_to (assigned_to, is_current)` : A **composite index** accelerating staff task resolution by directly filtering active assignments for an officer.

9.2 Eradicating the N+1 Query Problem via Eager Loading

> **The N+1 Anti-Pattern:** If the dashboard fetches 50 complaints and loops through them calling `$complaint->complainant->full_name` , Laravel executes 1 query for the complaints plus 50 individual queries for the users (51 queries total), causing severe database latency.

```
// Optimized Solution in Admin/ComplaintController.php using Eager Loading
$complaints = Complaint::with([
    'complainant',       // Pre-loads resident records in 1 query: WHERE user_id IN (...)
    'category',          // Pre-loads incident categories
    'status',            // Pre-loads status metadata
    'area',              // Pre-loads purok details
    'currentAssignment.assignee' // Pre-loads assigned officer
])->latest()->paginate(15); // Reduces 250+ queries to exactly 6 optimized batch queries!
```

10. HIGH-YIELD EXAM & CAPSTONE DEFENSE REVIEWER (QUESTIONS 1 TO 6)

These questions reflect the most common, challenging technical inquiries asked by university panels and Information Management professors during thesis defense:

Q1 Why did your team choose a Relational DBMS (MySQL/MariaDB) over a NoSQL document database like MongoDB?

Model Answer: Banlic-CCIRS models structured civic processes with strict relational dependencies (complaints, resident verifications, assignments, resolutions). An RDBMS provides ACID transaction guarantees, schema-level referential integrity (foreign keys), and prevents data anomalies through 3NF normalization. NoSQL is suited for unstructured or rapidly changing schemas, whereas barangay complaints require transactional consistency and non-repudiation.

Q2 Prove that your database schema satisfies Third Normal Form (3NF).

Model Answer: The schema satisfies 1NF because all attributes are atomic (no multi-valued columns or repeating groups). It satisfies 2NF because non-key attributes are fully functionally dependent on primary keys (verified in `area_staff` where assignment date depends on the composite key). It satisfies 3NF because all transitive dependencies are eliminated: lookup values (departments, categories, statuses, priorities) exist in independent reference tables rather than being duplicated inside the complaints table.

Q3 What is the role of an ORM like Eloquent, and does it replace SQL?

Model Answer: Eloquent acts as an Object-Relational Mapping bridge between PHP's object-oriented domain and MySQL's relational tables using the Active Record pattern. It abstracts manual SQL string composition, manages relationships, and provides mass-assignment protection. It does not replace SQL; instead, it compiles model operations into optimized, parameterized SQL queries executed via PDO.

Q4 What happens to child records when a complaint is deleted? Explain your ON DELETE rules.

Model Answer: Referential integrity rules govern parent-child relationships. If a complaint is deleted, foreign keys in `attachments`, `complaint_status_history`, `investigations`, and `resolutions` execute `ON DELETE CASCADE`, safely purging dependent data and preventing orphaned records. Conversely, foreign keys referencing users or departments utilize `ON DELETE SET NULL` or `RESTRICT` to protect historical accountability.

Q5 How does your system protect against SQL Injection attacks?

Model Answer: SQL Injection occurs when untrusted user input is concatenated directly into SQL command strings. Banlic-CCIRS prevents this by using Eloquent and Laravel's Query Builder, which rely exclusively on **PDO Prepared Statements**. Input parameters are transmitted separately from the SQL statement template, ensuring input is treated as raw data rather than executable code.

Q6 Why do you use reference_no in URLs (/complaints/BRGY-2026-00012) instead of complaint_id?

Model Answer: Exposing sequential internal auto-increment primary keys creates an **Insecure Direct Object Reference (IDOR)** vulnerability, allowing attackers to enumerate records. Using an unpredictable reference number implemented via Eloquent's `getRouteKeyName()` conceals internal database primary keys and provides a human-readable tracking code for residents.

10. HIGH-YIELD EXAM & CAPSTONE DEFENSE REVIEWER (QUESTIONS 7 TO 12)

Q7 How are database migrations superior to importing a raw .sql file?

Model Answer: Migrations provide version control for database schemas. They allow developers to track incremental schema modifications over time, roll back erroneous changes via the `down()` method, and ensure consistency across development, staging, and production environments without manual, error-prone SQL dump imports.

Q8 Where are database transactions implemented in your codebase, and what problem do they solve?

Model Answer: In `ComplaintController@store` and `Admin\ComplaintController@updateStatus`, updates are enclosed within `DB::transaction()`. When filing a complaint, if inserting file attachments fails after the main complaint record is created, the transaction issues an automatic `ROLLBACK`, preventing corrupted, partial records from persisting in the database.

Q9 How does your system ensure data confidentiality and resident privacy under Republic Act 10173?

Model Answer: The system applies privacy-by-design. Passwords are encrypted with salted bcrypt (cost 12), preventing exposure even during a database breach. Resident identities on public dispatch lists are dynamically masked (`getMaskedComplainantAttribute()`), and verification ID files are stored on isolated filesystem paths restricted by role middleware.

Q10 How does database indexing work under the hood in MySQL InnoDB?

Model Answer: InnoDB organizes table records in a **B+Tree Clustered Index** ordered by Primary Key. Secondary indexes (e.g., `status_id`, `area_id`) store the indexed column value alongside a pointer to the primary key. When filtering, MySQL traverses the B+Tree in O(log N) time to retrieve primary keys, bypassing slow, exhaustive full-table scans.

Q11 What is the significance of database transactions during complaint status reassignment?

Model Answer: Reassigning a case requires modifying three tables: deactivating the previous assignment (`is_current = false`), inserting the new assignment, and recording a status history event. Wrapping these in `DB::transaction()` ensures all three operations succeed together; a network failure cannot leave two active officers or missing timeline logs.

Q12 What are audit logs, and why are they critical for accountability in government information systems?

Model Answer: The `audit_logs` table maintains an immutable non-repudiation record of all administrative actions (CREATE, UPDATE, ASSIGN, STATUS_CHANGE). It captures the actor's user ID, IP address, target table, record ID, and timestamp, guaranteeing accountability, dispute traceability, and audit compliance for barangay governance.

Summary for Capstone Defense: When presenting your database architecture, emphasize that the database was engineered with **mathematical rigor (3NF)**, **bulletproof integrity (foreign key cascades & ACID transactions)**, and **enterprise integration (Laravel Eloquent ORM & PDO prepared statements)**.